SA2: Discussion & Comparison

Dela Cruz, Cloie Sofia R. | Zabala, Ralf Cedric T.

6.1 Hyperparameter Impact

Model A (AdamW, 25 epochs, batch 4, lr 0.01) used an adaptive optimizer with a relatively high learning rate and very small batch size. The small batch of 4 introduces high gradient noise, which can help escape local minima but also causes unstable training. Despite having the fewest epochs, AdamW's per-parameter adaptive learning rates allowed it to converge to a reasonably strong solution (mAP50 = 0.6034, F1 = 0.5895). However, the high precision (0.6682) but lower recall (0.5274) indicates the model became *conservative*—it made confident predictions but missed many objects.

Model B (SGD, 30 epochs, batch 20, lr 0.001) was the weakest performer across all metrics (mAP50 = 0.4858, F1 = 0.5018). SGD with a low learning rate of 0.001 and only 30 epochs was simply not enough for convergence. SGD requires careful learning rate scheduling and typically needs more epochs than adaptive optimizers to reach comparable performance. The larger batch size of 20 provides smoother gradients but, combined with the low learning rate, resulted in underfitting. Precision and recall were nearly balanced (~0.50 each), meaning the model neither committed strongly nor recalled well—it was undertrained.

Model C (auto optimizer → AdamW lr=0.000909, 40 epochs, auto-batch 19) achieved the best results across the board (mAP50 = 0.6178, F1 = 0.6192). The auto setting selected AdamW with an automatically tuned learning rate of ~0.000909 and batch size of 19. With 40 epochs—the longest training duration—the model had the most opportunity to learn. It achieved the best balance of precision (0.6500) and recall (0.5911), confirming that more training time with a well-tuned optimizer yields the strongest generalization.

Key takeaway: Optimizer choice and training duration mattered more than batch size alone. AdamW consistently outperformed SGD in this low-data regime (~1,010 training images), and the auto-tuned configuration demonstrated that Ultralytics' automatic hyperparameter selection is a strong default.

6.2 Results Interpretation

Model	mAP50	Precision	Recall	F1 Score
Model A	0.6034	0.6682	0.5274	0.5895
Model B	0.4858	0.5007	0.5029	0.5018
Model C	**0.6178**	**0.6500**	**0.5911**	**0.6192**

- **mAP50** measures overall detection accuracy at IoU ≥ 0.50. Model C leads at 0.6178, roughly 1.4 percentage points above Model A and 13.2 points above Model B. None of the models exceed 0.65, which indicates the task (7-class Philippine road vehicle detection) remains challenging at this dataset scale.
- **Precision** reflects how many of the model's detections were correct. Model A's precision (0.6682) is the highest, meaning it produced fewer false positives—but at the cost of missing many true objects (low recall). Model C balances this better.
- **Recall** measures how many ground-truth objects were found. Model C's recall of 0.5911 is the highest, meaning it detected ~59% of all real objects. Model A only detected ~53%, and Model B ~50%.
- **F1 Score** harmonically combines precision and recall. Model C's F1 of 0.6192 is the best, confirming it offers the most balanced detection performance overall.

From the confusion matrices:

- All three models show significant predictions falling into the **background** row (bottom row), meaning many real objects go undetected (false negatives). This is the dominant error mode across all configurations.
- Off-diagonal confusions are relatively sparse, suggesting that when the models *do* detect an object, they usually classify it into the correct category. The main challenge is detection sensitivity, not classification accuracy.

6.3 Per-Class Struggles

Class	Model A (F1)	Model B (F1)	Model C (F1)	Observation
Bicycle	0.549	0.548	0.583	Moderate. High precision but low recall across all

				models—many bicycles are missed, likely due to small size.
Car	0.589	0.517	0.615	Moderate. Some confusion with SUV (Model A CM shows 0.20 SUV predicted for true Car).
Jeepney	**0.746**	**0.645**	**0.730**	**Best class for all models.** Jeepneys have a distinctive shape and size, making them the easiest to detect.
Pedestrian	0.548	0.493	0.602	High precision but low recall—many pedestrians go undetected. Small object size and occlusion are likely causes.
SUV	0.615	0.534	0.661	Moderate. Confused with Car due to similar shape. Model C's CM shows SUV predicted for Car at 0.15.
Tricycle	**0.366**	**0.203**	**0.417**	**Worst class for all models.** Tricycle had the fewest training annotations (539) and is visually variable. Model B's F1 of 0.203 is essentially non-functional. Even Model C only reaches 0.417.

Truck	0.593	0.467	0.625	Moderate to good. Distinct shape helps, but relatively few samples (539 annotations).

Classes the models struggled with most:

1. **Tricycle** is by far the hardest class. Despite augmentation-based balancing in SA1, the imbalance ratio remained 2.93× (539 Tricycle vs. 1,581 Bicycle annotations). Tricycles also exhibit high visual variability (different body styles, coverings, and passenger configurations), making feature learning difficult. In Model B's confusion matrix, the Tricycle diagonal value was only 0.09—meaning the model almost entirely failed to detect them. Model C improved this to 0.35, but it remains the weakest class.
2. **Pedestrian** is the second most challenging class. Despite having 653 annotations, pedestrians are typically small, frequently occluded, and appear in varying poses. All models show recall below 0.50 for Pedestrian (Model A: 0.42, Model B: 0.44, Model C: 0.47), meaning more than half of all pedestrians are missed. In a real-world traffic monitoring application, this would be a critical safety concern.
3. **Bicycle** also suffers from low recall (~0.41–0.47) despite having the most annotations (1,581). This paradox likely occurs because many bicycle annotations are small or partially visible. The high annotation count includes many augmented copies that may not add sufficient visual diversity.

How the scores reflect these struggles:

- Low recall on Tricycle/Pedestrian/Bicycle directly pulls down the overall mAP50 and F1.
- The high background false-negative rates visible in all confusion matrices confirm that the models' primary failure mode is *missing objects* rather than *misclassifying* them.
- Jeepney's consistently high performance (F1 > 0.64 across all models) shows that distinctive, large objects with sufficient training data are well-handled even by the weakest configuration.

6.4 Recommendation

Model C is the recommended configuration for further development, as it achieves the highest mAP50 (0.6178), F1 (0.6192), and recall (0.5911). Its auto-tuned hyperparameters provide a strong baseline without manual tuning.

However, none of the models are deployment-ready. An mAP50 below 0.65 and recall below 0.60 mean that roughly 40% of road objects would be missed in a real-world scenario. To improve:

1. **Increase dataset size** - 1,010 training images is small. Collecting 3,000–5,000 images would significantly boost performance, especially for underrepresented classes like Tricycle.
2. **Targeted augmentation for Tricycle and Pedestrian** - Apply more aggressive augmentations (rotation, scale jitter, copy-paste) specifically for these classes.
3. **Train for more epochs** - Model C was still improving at epoch 40. Training for 80–100 epochs with early stopping (patience=20) could yield further gains.
4. **Use a larger model** - YOLOv26s or YOLOv26m would have more capacity to learn fine-grained features, at the cost of slower inference.
5. **Increase image resolution** - Using imgsz=1280 could help with small-object detection (Bicycle, Pedestrian), though it requires more GPU memory.

6.5 Conclusion

This activity trained and evaluated three YOLOv26n models with distinct hyperparameter configurations on a 7-class Philippine road vehicle detection dataset. The results demonstrate that:

- **Optimizer choice** has the largest impact: AdamW consistently outperformed SGD in this low-data regime.
- **Training duration** matters: Model C (40 epochs) outperformed Model A (25 epochs) despite similar optimizer families.
- **Auto-tuned hyperparameters** (Model C) produced the best overall results, validating Ultralytics' automatic configuration as a strong default.
- **Tricycle and Pedestrian** remain the most challenging classes due to visual variability, small size, and relative data scarcity.
- All models are not yet deployment-ready but provide a solid foundation for further improvement through data collection, augmentation, and extended training.